

Bloc A · BLE

1. Expliqueu la diferència entre GAP i GATT. Quin dels dos governa que un dispositiu sigui visible i quin governa que se'n puguin llegir dades?

El GAP es l'encarregat de la visibilitat, connexió... El GATT es el que s'encarrega de l'organització i la transferència de dades.

2. Un perifèric BLE pot ser simultàniament servidor GATT i client GATT? Justifiqueu-ho.

Si, ja que pot oferir les seves dades com la temperatura, i al mateix temps llegir dades d'un altre dispositiu.

3. Descriviu la jerarquia servei → característica → descriptor amb un exemple inventat: un node que publica temperatura i que accepta una consigna.

Servei → Agrupa funcionalitats relacionades → Servei de control de clima

Característica → Dades concretes que es poden llegir, escriure o notificar → Temperatura actual

Descriptor → Informació adicional sobre la característica → Les unitats de mesura (°C)

4. Per què existeixen UUID de 16 bits i de 128 bits? Quan heu de fer servir cadascun al vostre projecte?

De 16 bits estan reservats per la Bluetooth SIG, per a serveis i característiques estàndar.

Els de 128 bits s'utilitzen per aplicacions pròpies, per evitar conflictes.

Utilitzarem els de 128 per tot lo que no sigui estàndar

5. Quina és la MTU per defecte d'una connexió BLE i quants bytes útils queden per a una notificació? Què passa si voleu enviar una contrasenya Wi-Fi de 40 caràcters?

La MTU per defecte es de 23 bytes amb 20 bytes útils

Que supera els 20 bytes i per tant no es pot enviar sencera.

Bloc B · Provisioning

1. Un node acabat de sortir de fàbrica no sap ni el nom ni la clau de la vostra Wi-Fi.

Enumereu tres maneres diferents de fer-l'hi arribar i digueu quin problema té cadascuna.

Hardcoded → No és flexible, al mínim canvi de router, contrasenya... cal tornar a reflashar el firmware

Connexió per cable sèrie (UART / USB) → Requereix un ordinador, cables...

Teclat i pantalla al dispositiu → Molt més costos

2. Compareu el provisioning per SoftAP (el node aixeca un punt d'accés temporal) amb el provisioning per BLE: experiència d'usuari, seguretat i cost en memòria.

SoftAP → Cal canviar el wifi del mòbil i connectarse a la xarxa creada, poc intuïtiu. Les xarxes no tenen xifratge inicial. Consumeix molta memòria RAM i Flash

BLE → El mòbil manté la connexió wifi i es comunica en segon pla amb el microcontrolador. Permet xifratge i més seguretat. Es molt més lleguer i no requereix aixecar serveis de xarxa IP complexos

3. Què és la NVS de l'ESP32 i en què es diferencia d'escriure en una variable global? Què li passa a la NVS quan reflasheu el firmware?

NVS → Memòria Flash no volàtil. Aquesta no resideix a la RAM i aguanta un reinici. Si es reflashea les dades es mantenen.

4. Dissenyeu sobre el paper la màquina d'estats del vostre node: quins estats hi ha des que s'engega fins que està publicant dades, i quins esdeveniments provoquen cada transició. Dibuixeu-la.

Estats principals:

1. **INIT**: S'engega el node i es llegeixen les credencials de la NVS.
2. **PROVISIONING_BLE**: S'activa el BLE (*Advertising*) si no hi ha credencials o falla la connexió.
3. **CONNECTING_WIFI**: Intenta connectar-se al Punt d'Accés.
4. **RUNNING**: Apaga el BLE (per estalviar ràdio/recursos) i comunica amb el Gateway per HTTP/REST.

Transicions i esdeveniments:

- **INIT** → **CONNECTING_WIFI**: *Credencials trobades a la NVS.*
- **INIT** → **PROVISIONING_BLE**: *NVS buida o botó de reset premut.*
- **PROVISIONING_BLE** → **CONNECTING_WIFI**: *Credencials rebudes per BLE.*
- **CONNECTING_WIFI** → **RUNNING**: *Connexió Wi-Fi exitosa.*
- **CONNECTING_WIFI** → **PROVISIONING_BLE**: *Error de connexió o timeout.*

5. Com ha de tornar el node a l'estat de configuració si l'usuari canvia de router?

Proposeu-ne un mecanisme que un usuari no tècnic pugui fer servir.

Mantenir premut un botó físic del dispositiu durant 5 segons. L'ESP32 detecta aquesta pulsació, esborra les credencials guardades a la NVS i es reinicia en mode **PROVISIONING_BLE** per tal que l'usuari pugui introduir la nova Wi-Fi des de l'aplicació del mòbil.

Bloc C · REST

1. Digueu què vol dir que una operació és idempotent i classifiqueu GET, POST, PUT i DELETE.

Una operació idempotent es una operació que produeix el mateix resultat al servidor independentment de quantes vegades s'executi consecutivament amb les mateixes dades.

GET: Idempotent (només llegeix dades, no modifica l'estat).

PUT: Idempotent (reemplaça o estableix un recurs exactament amb el valor enviat).

DELETE: Idempotent (esborrar un recurs un cop o deu vegades deixa el recurs esborrat).

POST: NO idempotent (cada petició pot crear un nou recurs o generar efectes secundaris addicionals).

2. Quins codis d'estat retornaríeu en aquests tres casos: consulta correcta, consigna fora de rang, node encara no configurat?

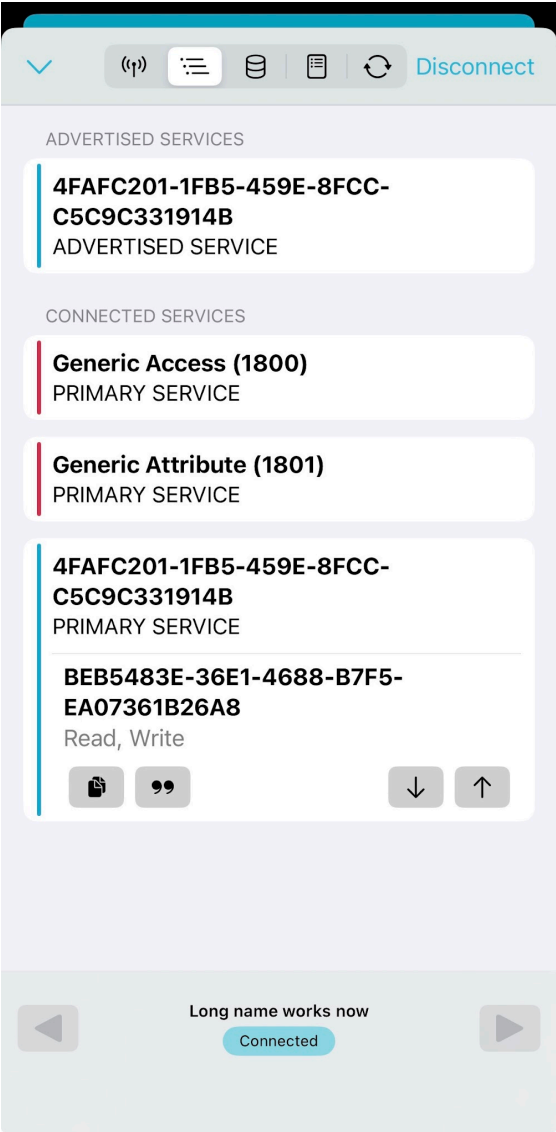
Consulta correcta → 200 OK

Consigna fora de rang → 400 Bad Request

Node encara no configurat → 503 Service Unavailable

3. Per què una API REST sobre HTTP és una mala idea per a telemetria contínua d'un node a bateria, i en canvi és raonable per a configuració i diagnòstic puntual?

Per a una telemetria contínua és una mala idea perquè HTTP requereix establir connexions TCP/TLS complexes i enviar capçaleres molt pesades. Això fa que es gasti molt més ràpid la bateria. Per a configuració i diagnòstic puntual es raonable perquè es tracta d'operacions ocasionals.



```
-----  
1: WSonnet0224 | Canal: 3 | RSSI: -50 dBm  
2: Garatge | Canal: 1 | RSSI: -89 dBm  
3: WiFi-AP07631029 | Canal: 1 | RSSI: -97 dBm  
-----
```